Improving code vulnerability detection using LLMs

Andrei Tudor Fărcăşanu
Department of Computer Science and Engineering
National University of Science and Technology POLITEHNICA Bucharest
tudor.farcasanu@stud.acs.upb.ro

Traian Rebedea
Department of Computer Science and Engineering
National University of Science and Technology POLITEHNICA Bucharest
traian.rebedea@upb.ro

Abstract

Large Language Models (LLMs) show promise for automated vulnerability detection but struggle to distinguish between vulnerable code and its patched counterpart. This research advances LLM-based vulnerability detection through a multi-stage pipeline combining dataset consolidation, reasoning-augmented fine-tuning, and reinforcement learning. We merge three code vulnerability datasets (PrimeVul, SVEN, SecVulEval) into a new dataset containing 12,912 unique deduplicated functions through a function-level deduplication strategy, then use Qwen3-32B as a teacher model to generate reasoning-augmented training data that teaches the student model not just the classification outcome but the analytical process leading to it. Supervised fine-tuning of Qwen3-4B on this dataset, followed by Group Relative Policy Optimization (GRPO) with a tiered reward structure evaluating both classification accuracy and vulnerable line localization, achieves a Pair-wise Correct score of 28.38% on a combined benchmark of 970 vulnerability pairs and 25.93% on the PrimeVul test split, a $\sim$100% relative improvement over GPT-4's reported performance in PrimeVul [1]. The GRPO model also surpasses all SFT-only variants, the best of which reaches 25.51% P-C, confirming that reinforcement learning provides a measurable gain beyond supervised fine-tuning alone. Our experiments further reveal that hint-free training prompts produce stronger discriminative models at the cost of output parsability, while iterating on the GRPO reward structure proves critical for controlling undesirable behaviours such as line prediction spraying.

1 Introduction

The escalating complexity of modern software systems has been paralleled by a surge in security vulnerabilities, making automated detection a critical field of research. While traditional static and dynamic analysis tools have been foundational, they often struggle with high false positive rates or require extensive manual configuration. The advent of Large Language Models (LLMs) has introduced a new paradigm, demonstrating a remarkable ability to understand and generate code, which has naturally been extended to vulnerability detection [16, 4].

Despite their potential, recent work has shown that LLMs often fail to distinguish between vulnerable code and its subtly patched counterpart. The PrimeVul benchmark [1] showed that even GPT-4 achieves only 12.94% Pair-wise Correct (P-C) score, meaning it correctly classifies both samples in a vulnerability pair less than 13% of the time. This suggests that models may overfit to superficial textual features rather than grasping the underlying semantic causes of vulnerabilities.

Inspired by knowledge-level enhancement frameworks like Vul-RAG [2], we hypothesize that an LLM's discriminative ability can be significantly improved by fine-tuning it on data that explicitly articulates the reasoning behind a vulnerability's existence and its corresponding fix. We approach this through a multi-stage methodology: consolidating multiple vulnerability datasets, generating reasoning-augmented training data via a teacher model, supervised fine-tuning of a smaller student model, and applying reinforcement learning with targeted reward signals.

3rd Best Romanian AI Thesis Awards - Bucharest, 2026

Main contributions:

- A function-level deduplication pipeline that merges three complementary vulnerability datasets into 12,912 unique training functions, recovering thousands of pairs lost by prior commit-level strategies.
- A teacher-student pipeline where Qwen3-32B generates reasoning-augmented annotations through an iteratively refined prompt design that balances metadata richness with preventing reasoning dilution.
- A GRPO training framework with tiered reward structures evaluating both classification and line localization, achieving 28.38% P-C on a combined benchmark of 970 vulnerability pairs and 25.93% on the PrimeVul test split, a $\sim$100% relative improvement over GPT-4 as reported in PrimeVul [1].
- An ablation study across five SFT variants and three GRPO reward configurations, revealing the trade-off between hint-free prompts training (stronger discrimination) and hint-rich prompts training (better output parsability).

2 Related Work

Vulnerable vs. Patched Code Discrimination. The PrimeVul benchmark [1] introduced paired vulnerable/fixed functions with pair-wise metrics (P-C, P-V, P-B, P-R) to measure discrimination. Their results showed even GPT-4 struggles to correctly classify both samples in a pair, establishing the key challenge our work addresses.

Real-World Vulnerability Analysis. CyberGym [11] demonstrates real-world complexity through a large-scale evaluation of 1,507 vulnerabilities, where even top AI agents achieve only 11.9% success rate in generating proof-of-concept tests, while discovering 35 zero-day vulnerabilities. JitVul [12] addresses Just-In-Time vulnerability detection, showing that LLM-based agents with reasoning capabilities outperform isolated function-level analysis.

Explainability and Foundational Methods. Recent work on Explainable Vulnerability Detection [3] using edge-aware graph attention networks bridges black-box predictions and actionable insights. Graph Neural Networks on ASTs and CFGs [15], systems like VulDeePecker [5] and SySeVR [6], and dataflow-inspired approaches [10] provide foundational techniques that inform our understanding of what semantic patterns are critical for detection, suggesting that enhancing LLMs with structured reasoning is a promising direction.

3 Approach

Our pipeline consists of four stages: dataset consolidation, teacher annotation, supervised fine-tuning (SFT), and Group Relative Policy Optimization (GRPO).

3.1 Dataset Consolidation

We merge three data sources into a unified dataset: PrimeVul (paired vulnerable/patched functions with CVE metadata), SVEN (1,606 manually-verified functions with 94% claimed label accuracy), and SecVulEval (additional CVE/CWE-linked samples with line-level changes). Our function-level deduplication pipeline groups samples by `(commit_id, func_name)` and applies a three-tier conflict resolution strategy: (1) identical code hashes are collapsed by source priority (SVEN $>$ SecVulEval $>$ PrimeVul) with metadata merging; (2) different hashes within the same group undergo pairwise resolution based on code similarity, diff size, and file paths; (3) surviving variants are cross-enriched with missing CVE/CWE metadata from siblings (more details available in main thesis). This recovers thousands of pairs previously lost by commit-level collapsing, yielding **12,912 unique deduplicated functions** covering 6,208 unique CVEs and 144 CWE categories.

3.2 Teacher Model Annotation

We use Qwen3-32B to generate reasoning traces for each sample. The prompt design evolved through five iterations, balancing information density with reasoning trace quality. Key iterations include: (T2) introducing a JSON output schema with CVE identifiers added to the metadata context; (T3) unifying the JSON schema to output *important lines* instead of just vulnerable ones, handling both vulnerable and patched samples to also allow scoring for patched samples during GRPO, and adding dynamically truncated commit messages; and (T5) dropping commit messages when a CVE description is present, after observing the teacher devoting excessive reasoning to paraphrasing metadata rather than analysing code. Each prompt includes line-numbered code and optional ground-truth hints about changed lines, with the target output being a structured JSON containing classification, important line numbers, and a reasoning summary.

3.3 Supervised Fine-Tuning

We fine-tune Qwen3-4B using the teacher-generated annotations. Five SFT variants were tested, varying whether hints (CVE context, changed lines) appear in the training prompts. The key finding is a trade-off: *hint-free SFT* (where training prompts contain only raw code despite responses being generated with full context) produces models with stronger discrimination but lower JSON parsability (the model often gets stuck reasoning and fails to output valid JSON), while *hints-in-SFT* models exhibit better format compliance but inherit the teacher's vulnerability-prediction bias. All Qwen3-4B SFT runs use full-parameter fine-tuning with DeepSpeed ZeRO-3 [8] across 3×H100 GPUs, learning rate 2×10^{-5}, and 1 epoch.

3.4 Group Relative Policy Optimization

GRPO [9] refines the SFT checkpoint using reinforcement learning without a separate value network. For each prompt, it generates G=16 candidate responses, computes per-sample rewards, and estimates advantages within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, \ldots, r_G\})}{\text{std}(\{r_1, \ldots, r_G\})} \tag{1}$$

We tested three reward configurations: (G1) a 4-tier structure (1.0/0.6/0.3/0.05/0.0) scoring both vulnerable and patched samples; (G2) a 5-tier structure adding a 1.25 high-precision tier for $\geq$75% line overlap; and (G3) an adjusted 4-tier (1.0/0.75/0.3/0.1/0.0) with two data quality guards: a *spray guard* that withholds localization rewards when the model predicts more than 60 lines (set slightly higher than the figures the 99th percentile returned), and *GT line capping* that zeroes out ground truth for outlier samples (99th percentile line count). G3 also disables patched sample scoring, since the model cannot logically deduce which lines were added in a patch. All runs use β=0 (no KL penalty), following recent findings [13, 7]. The three runs also differ in their SFT starting checkpoint and rollout temperature (1.0 for G1, 0.975 for G2 and G3).

4 Experimental Analysis

4.1 Setup

Evaluation is conducted on a combined test set of 970 vulnerability pairs (1,940 samples): PrimeVul test (407 pairs), PrimeVul validation (354 pairs), and SVEN validation (209 pairs), which weren't used during training. All results are reported at temperature T=0.6, which generally yielded the best P-C scores across variants. Pair-wise metrics measure the model's ability to correctly classify *both* samples in a pair: P-C↑ (both correct), P-V↓ (both labeled vulnerable), P-B↓ (both labeled benign), P-R↓ (labels swapped).

4.2 Results

Table 1 presents the main results. For models evaluated across multiple runs, we report the metrics from the run achieving the best P-C, alongside the P-C average and maximum spread in the format: Best (Avg±Spread).

4.3 Ablation: SFT Hint Configuration

Among SFT-only models, hint-free training (where prompts contain only raw code, while the teacher response was generated with full context) achieves competitive P-C (23.25%) but at the cost of the highest P-B (44.48%) and lowest parsability (1,696/1,940). The model becomes conservative, often failing to complete reasoning within the token limit. Re-introducing hints restores parsability (1,842/1,940) and achieves the highest F1 (59.95%), but inherits the teacher's vulnerability-prediction bias (P-V of 45.10%). The final variant with refined teacher annotations (dropping commit messages when CVE descriptions are present) strikes a middle ground: 24.28% P-C with balanced P-V/P-B and strong parsability (1,870/1,940).

4.4 Ablation: GRPO Reward Structure

GRPO Run 1, starting from the hint-free SFT base with the original 4-tier rewards, achieves the highest P-C (28.38%) but also exhibits the highest variance (±4.81 spread) and lowest parsability (1,693/1,940). Run 2's 5-tier structure achieved higher absolute rewards during training (∼0.55–0.65 vs. ∼0.40–0.45 for Run 1), but this did not translate to better discrimination; instead, rollout analysis revealed it incentivised the model to "spray" many predicted lines to hit the 75% overlap threshold. Run 3's data quality guards successfully controlled this behaviour, maintaining excellent

Table 1: Evaluation Results. Best P-C per section in **bold**.

Model	Acc↑	F1↑	P-C(%)↑	P-V(%)↓	P-B(%)↓	P-R(%)↓	Parsable
Combined Test Set (970 pairs)							
Baselines							
Qwen3-4B (Base)	52.57	52.20	20.68 (20.27±0.83)	30.92	32.91	15.46	1923/1940
Qwen3-32B (Teacher)	54.22	59.31	22.39 (22.39 ± 0.00)	44.47	19.16	13.95	1927/1940
SFT Models (Qwen3-4B, full fine-tuning)							
SFT: Hints-in (T2, commit-dedup)	54.87	58.23	25.51	37.47	20.57	16.43	1815/1876
SFT: Hint-free (T3)	51.23	41.71	23.25 (22.30±1.90)	11.96	44.48	20.29	1696/1940
SFT: Hints re-introduced (T4)	54.45	59.95	23.12 (23.03±0.19)	45.10	17.88	13.89	1842/1940
SFT: Refined hints (T5)	54.01	57.92	24.28 (23.05±2.04)	39.18	20.86	15.67	1870/1940
GRPO Models (Qwen3-4B)							
GRPO Run 1: 4-tier, hint-free SFT	56.11	49.49	**28.38** (25.78±4.81)	13.92	40.56	17.13	1693/1940
GRPO Run 2: 5-tier, hints-in SFT	54.09	58.11	24.60 (22.82±3.48)	39.14	19.79	16.44	1856/1940
GRPO Run 3: Adj. 4-tier + guards	55.57	58.37	24.25 (23.68±1.55)	38.09	24.58	13.06	1875/1940
PrimeVul Test Split Only (407 pairs)							
PrimeVul GPT-4 (Reported) [1]	–	–	12.94	54.26	24.47	8.33	N/A
Qwen3-8B LoRA	52.80	42.70	18.67	14.74	52.58	14.00	N/A
GRPO Run 1 (best model)	55.44	48.01	**25.93** (22.74±6.39)	15.31	44.06	14.68	734/848

parsability (∼1,875/1,940) and the most balanced P-V/P-B distribution, though P-C improvement over its SFT base was modest.

5 Discussion

Our best model achieves 28.38% P-C on the combined test set and 25.93% on the PrimeVul test split, the latter representing a 100% relative improvement over GPT-4's reported 12.94% and a 39% improvement over our initial LoRA baseline (18.67%). The GRPO model also surpasses all SFT-only variants, confirming that reinforcement learning provides a measurable gain beyond supervised fine-tuning.

A central finding is the trade-off between discrimination and parsability. The hint-free pipeline produces the strongest discriminative signal by forcing the model to reason from code alone, but at the cost of output stability, with the model frequently failing to finish reasoning and sometimes failing to produce valid JSON. Hints-in-SFT models are more reliable but inherit the teacher's bias toward predicting vulnerabilities (P-V $> 40\%$). This suggests that the optimal approach may involve hint-free GRPO training with a reward component specifically targeting format compliance.

The teacher model (Qwen3-32B) achieves only 22.39% P-C, confirming that fine-tuned student models genuinely surpass the teacher's zero-shot capability through the distillation and RL process. Interestingly, the SFT prompt format shapes the student's default behaviour at evaluation time (when no hints are present): hints-in-SFT models, trained with explicit vulnerability cues, become reliant on those cues and over-predict vulnerabilities when they are absent (elevated P-V), while hint-free SFT models, never exposed to metadata context, become conservative and default to benign predictions (high P-B).

The GRPO reward structure proved critical for training dynamics. The 5-tier reward (G2), despite yielding higher absolute training rewards, inadvertently taught the model to exploit the localization component by predicting excessive line numbers. The introduction of spray guards and GT line capping in G3 successfully mitigated this, showing that reward engineering requires careful attention to potential exploitation strategies.

6 Conclusion

We demonstrated that reasoning-augmented fine-tuning combined with GRPO reinforcement learning substantially improves vulnerable/patched code discrimination, achieving a 100% relative improvement over GPT-4 on PrimeVul. Key directions for future work include continuous reward functions for finer gradient signals, integration of static analysis tool outputs into the reward function, newer code-specialised foundation models, and extending beyond isolated functions to repository-level context incorporating call graphs and data-flow information, as motivated by benchmarks like CyberGym [11] and JitVul [12].

7 Limitations

Our evaluation relies on the PrimeVul benchmark, which focuses on C/C++ functions in open-source projects, limiting generalizability to other languages and potentially proprietary codebases. The 4B parameter student model may lack capacity for more complex vulnerability patterns. All P-C scores exhibit notable run-to-run variance (up to ± 6.39 on PrimeVul), making definitive ranking between configurations difficult. The best model's parsability (1,693/1,940) means approximately 12.7% of responses are invalid JSON, limiting practical deployment. Finally, our reward function relies on line-level overlap with ground truth, which may not capture semantic correctness of vulnerability localization.

8 AI Transparency Statement

- **Use of AI in methodology:** The core research pipeline uses Qwen3-32B (Alibaba) as a teacher model to generate reasoning-augmented training annotations, and Qwen3-4B and Qwen3-8B as the student models fine-tuned via SFT and GRPO. SGLang [14] was used for model inference during dataset generation and evaluation.
- **Use of AI in writing:** Artificial intelligence tools (Claude 4.6, Anthropic) were used only to assist with language refinement, and improving the clarity and consistency of the manuscript.

Acknowledgments

This work was conducted as part of a Master's thesis at the National University of Science and Technology POLITEHNICA Bucharest, supervised by Conf. Traian Rebedea. Training was performed on the UPB HPC cluster ($3\times$H100 GPUs). Models and datasets are publicly available at `https://huggingface.co/Andrefty`.

References

[1] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. Vulnerability detection with code language models: How far are we?, 2024. URL `https://arxiv.org/abs/2403.18624`.

[2] Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag, 2024. URL `https://arxiv.org/abs/2406.11147`.

[3] Radowanul Haque, Aftab Ali, Sally McClean, and Naveed Khan. Explainable vulnerability detection in c/c++ using edge-aware graph attention networks, 2025. URL `https://arxiv.org/abs/2507.16540`.

[4] Yue Li, Xiao Li, Hao Wu, Yue Zhang, Xiuzhen Cheng, Yating Liu, Fengyuan Xu, and Sheng Zhong. If llms would just look: Simple line-by-line checking improves vulnerability localization, 2025. URL `https://arxiv.org/abs/2410.15288`.

[5] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. Vuldeepecker: A deep learning-based system for vulnerability detection. In *Proceedings 2018 Network and Distributed System Security Symposium*, NDSS 2018. Internet Society, 2018. doi: 10.14722/ndss.2018.23158. URL `http://dx.doi.org/10.14722/ndss.2018.23158`.

[6] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing*, 19(4):2244–2258, July 2022. ISSN 2160-9209. doi: 10.1109/tdsc.2021.3051525. URL `http://dx.doi.org/10.1109/TDSC.2021.3051525`.

[7] Yiping Liu, Zijian Li, Michael Hall, Percy Liang, and Tatsunori Hashimoto. Understanding r1-zero-like training: A critical perspective, 2025. URL `https://arxiv.org/abs/2503.20783`.

[8] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020. URL `https://arxiv.org/abs/1910.02054`.

[9] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL `https://arxiv.org/abs/2402.03300`.

[10] Benjamin Steenhoek, Hongyang Gao, and Wei Le. Dataflow analysis-inspired deep learning for efficient vulnerability detection, 2023. URL `https://arxiv.org/abs/2212.08108`.

[11] Zhun Wang, Tianneng Shi, Jingxuan He, Matthew Cai, Jialin Zhang, and Dawn Song. Cybergym: Evaluating ai agents' real-world cybersecurity capabilities at scale, 2025. URL `https://arxiv.org/abs/2506.02548`.

[12] Alperen Yildiz, Sin G. Teo, Yiling Lou, Yebo Feng, Chong Wang, and Dinil M. Divakaran. Benchmarking llms and llm-based agents in practical vulnerability detection for code repositories, 2025. URL `https://arxiv.org/abs/2503.03586`.

[13] Qiying Yu, Zheng Zhang, Ruofei Wan, Xin Xing, and Baichun Wu. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL https://arxiv.org/abs/2503.14476.

[14] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs, 2024. URL https://arxiv.org/abs/2312.07104.

[15] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, 2019. URL https://arxiv.org/abs/1909.03496.

[16] Arastoo Zibaeirad and Marco Vieira. Vulnllmeval: A framework for evaluating large language models in software vulnerability detection and patching, 2024. URL https://arxiv.org/abs/2409.10756.